

Perfecto. Tomamos esta pausa como una **explicación estratégica completa** de cómo trabajaremos MongoDB cuando terminemos PostgreSQL.

La parte de MongoDB no será una base aparte “porque sí”. En este proyecto MongoDB tiene un propósito muy claro: guardar la información clínica variable, porque el historial médico no siempre tiene la misma estructura entre especialidades. El enunciado indica que PostgreSQL se usa para los datos transaccionales y MongoDB para datos flexibles y de alto volumen, especialmente historiales clínicos variables.

Objetivo de MongoDB en nuestro proyecto

MongoDB almacenará la parte clínica flexible del sistema:

- historial clínico de consultas atendidas
- signos vitales
- diagnósticos
- medicamentos recetados
- exámenes solicitados
- notas médicas
- campos específicos según especialidad

No guardaremos en MongoDB facturas, pagos, pacientes completos, médicos completos ni citas completas. Esos datos ya pertenecen a PostgreSQL porque necesitan integridad fuerte, relaciones y transacciones.

MongoDB se usará como una extensión clínica del sistema, conectada lógicamente con PostgreSQL mediante referencias como:

- id_cita
- id_paciente
- id_medico
- id_especialidad

Decisión principal de diseño

Usaremos una sola colección principal:

historiales_clinicos

Cada documento representará una consulta médica atendida.

No haremos muchas colecciones pequeñas como `diagnosticos`, `medicamentos`, `signos_vitales`, etc., porque en MongoDB nos conviene embeber esa información dentro del historial clínico.

La decisión será:

Un documento = una atención médica registrada por el médico.

Esto es defendible porque el historial clínico se consulta normalmente como una unidad completa: motivo, signos vitales, diagnóstico, receta, exámenes y notas.

Estructura esperada del documento MongoDB

El documento base será así:

```
{
  "id_cita": 15,
  "id_paciente": 3,
  "id_medico": 2,
  "id_especialidad": 1,
  "especialidad": "Cardiología",
  "fecha_consulta": "2026-06-06T10:00:00.000Z",

  "motivo_consulta": "Dolor en el pecho y dificultad para respirar",

  "signos_vitales": {
    "presion_sistolica": 120,
    "presion_diastolica": 80,
    "frecuencia_cardiaca": 78,
    "temperatura": 36.7,
    "peso_kg": 70,
    "altura_cm": 170
  },

  "diagnosticos": [
    {
      "codigo": "I10",
      "descripcion": "Hipertensión esencial",
      "tipo": "principal"
    }
  ],
}
```

```
"medicamentos": [  
  {  
    "nombre": "Losartán",  
    "dosis": "50 mg",  
    "frecuencia": "Cada 24 horas",  
    "duracion": "30 días"  
  }  
],  
  
"exámenes_solicitados": [  
  {  
    "nombre": "Electrocardiograma",  
    "prioridad": "normal",  
    "observaciones": "Realizar control cardiológico"  
  }  
],  
  
"notas_adicionales": "Paciente estable al finalizar la consulta.",  
  
"datos_especialidad": {  
  "riesgo_cardiovascular": "moderado",  
  "antecedentes_familiares": true  
},  
  
"created_at": "2026-06-06T10:30:00.000Z",  
"updated_at": null  
}
```

Por qué esta estructura es correcta

La parte fija del documento será:

id_cita
id_paciente
id_medico
id_especialidad
especialidad
fecha_consulta
motivo_consulta
signos_vitales
diagnosticos

medicamentos
exámenes_solicitados
notas_adicionales
created_at
updated_at

La parte flexible será:

datos_especialidad

Ese campo nos permite guardar datos diferentes según la especialidad.

Por ejemplo, en Cardiología:

```
"datos_especialidad": {  
  "riesgo_cardiovascular": "moderado",  
  "antecedentes_familiares": true,  
  "dolor_toracico": true  
}
```

En Dermatología:

```
"datos_especialidad": {  
  "tipo_lesion": "mancha",  
  "zona_afectada": "brazo derecho",  
  "tiempo_evolucion": "2 semanas"  
}
```

En Pediatría:

```
"datos_especialidad": {  
  "peso_percentil": 65,  
  "talla_percentil": 70,  
  "vacunas_al_dia": true  
}
```

Así justificamos el uso de MongoDB: no todas las especialidades registran exactamente lo mismo.

Cómo conectaremos MongoDB con PostgreSQL

MongoDB no tendrá claves foráneas reales como PostgreSQL. La relación será lógica.

El documento en MongoDB guardará IDs provenientes de PostgreSQL:

id_cita
id_paciente
id_medico
id_especialidad

La regla será:

Antes de insertar un historial clínico en MongoDB, la API debe validar en PostgreSQL que la cita exista y esté en estado atendida.

Esto cumple la regla:

El historial clínico solo puede registrarse para citas en estado atendida.

Esa regla no la pondremos directamente en MongoDB porque depende del estado real de una cita que vive en PostgreSQL.

Qué haremos al iniciar MongoDB

Cuando lleguemos a esa parte, trabajaremos así:

Nivel MongoDB 1: Crear base de datos
Nivel MongoDB 2: Crear colección historiales_clinicos
Nivel MongoDB 3: Definir schema con Mongoose
Nivel MongoDB 4: Crear función JavaScript reutilizable
Nivel MongoDB 5: Insertar historiales clínicos de prueba
Nivel MongoDB 6: Crear índices
Nivel MongoDB 7: Crear pipelines de aggregation
Nivel MongoDB 8: Probar reportes clínicos
Nivel MongoDB 9: Documentar decisiones de diseño
Nivel MongoDB 10: Integrar con API Express

Nivel MongoDB 1 — Crear base de datos

La base se llamará:

clinica_privada_mongo

La colección principal será:

historiales_clinicos

No crearemos varias colecciones al inicio porque el proyecto pide justificar embebido vs referencias. Nuestra justificación será:

Se embeben diagnósticos, medicamentos, exámenes y signos vitales porque pertenecen directamente a una consulta médica y normalmente se consultan junto con el historial.

Nivel MongoDB 2 — Crear colección

Crearemos:

```
db.createCollection("historiales_clinicos")
```

Pero no nos quedaremos solo con eso. También definiremos una estructura con Mongoose para la API.

Nivel MongoDB 3 — Schema con Mongoose

Usaremos Mongoose porque el enunciado lo permite para MongoDB. La idea no será ocultar los pipelines, sino ayudarnos a validar inserciones desde Node.js.

El schema conceptual tendrá:

id_cita: Number

id_paciente: Number

id_medico: Number

id_especialidad: Number

especialidad: String

fecha_consulta: Date

motivo_consulta: String
signos_vitales: Object
diagnosticos: Array
medicamentos: Array
exámenes_solicitados: Array
notas_adicionales: String
datos_especialidad: Object
created_at: Date
updated_at: Date

Esto será flexible, pero no desordenado.

Nivel MongoDB 4 — Función JavaScript reusable

El proyecto exige:

1 función JavaScript reusable para procesamiento o normalización de datos antes de inserción

Nuestra función se llamará:

normalizarHistorialClinico()

Esta función hará tareas como:

limpiar espacios innecesarios
normalizar nombres de medicamentos
asegurar arrays vacíos si no hay medicamentos o exámenes
convertir fecha a Date
normalizar texto de especialidad
validar estructura mínima

Ejemplo conceptual:

```
function normalizarHistorialClinico(data) {  
  return {  
    ...data,  
    especialidad: data.especialidad.trim(),  
    motivo_consulta: data.motivo_consulta.trim(),  
    diagnosticos: data.diagnosticos || [],  
  };  
}
```

```
medicamentos: data.medicamentos || [],
exámenes_solicitados: data.exámenes_solicitados || [],
created_at: new Date()
};
}
```

Esta función será muy importante para defender que no insertamos datos clínicos “sucios” directamente.

Nivel MongoDB 5 — Datos de prueba

El enunciado exige al menos:

150 historiales clínicos en MongoDB

Entonces generaremos historiales distribuidos entre especialidades.

La distribución sugerida será:

- 30 Medicina General
- 30 Cardiología
- 30 Pediatría
- 30 Dermatología
- 30 Ginecología o Traumatología

Cada historial deberá referenciar una cita atendida en PostgreSQL.

Eso significa que antes de insertar los 150 historiales, en PostgreSQL debemos tener suficientes citas con estado:

atendida

Nivel MongoDB 6 — Índices

Crearemos índices para acelerar consultas clínicas frecuentes.

Índices propuestos:

```
db.historiales_clinicos.createIndex({ id_paciente: 1, fecha_consulta: -1 })
```



```
db.historiales_clinicos.createIndex({ id_medico: 1, fecha_consulta: -1 })
db.historiales_clinicos.createIndex({ id_especialidad: 1 })
db.historiales_clinicos.createIndex({ "diagnosticos.descripcion": 1 })
db.historiales_clinicos.createIndex({ "medicamentos.nombre": 1 })
```

Por qué:

id_paciente + fecha_consulta:
para consultar historial clínico completo de un paciente en orden cronológico.

id_medico + fecha_consulta:
para revisar atenciones por médico.

id_especialidad:
para reportes clínicos agrupados por especialidad.

diagnosticos.descripcion:
para top diagnósticos frecuentes.

medicamentos.nombre:
para medicamentos más recetados.

Nivel MongoDB 7 — Pipelines de aggregation

El proyecto exige:

4 pipelines de aggregation
al menos uno usando \$facet

Nuestros 4 pipelines serán:

Pipeline 1: Top 5 diagnósticos más frecuentes

Responderá:

RC-07. Top 5 diagnósticos más frecuentes del último trimestre, agrupados por especialidad.

Usará:

\$match
\$unwind diagnósticos
\$group
\$sort
\$limit

Resultado esperado:

especialidad
diagnostico
cantidad

Pipeline 2: Medicamentos más recetados por especialidad

Responderá:

RC-08. Medicamentos más recetados por especialidad, con cantidad de prescripciones y porcentaje del total.

Usará:

\$unwind medicamentos
\$group
\$setWindowFields o cálculo por agrupación
\$project
\$sort

Resultado esperado:

especialidad
medicamento
cantidad
porcentaje

Pipeline 3: Análisis de signos vitales por grupo etario

Responderá:

RC-09. Análisis de signos vitales por grupo etario.

Este pipeline requiere edad. Como la fecha de nacimiento está en PostgreSQL, tendremos dos caminos:

Para el proyecto, lo más eficiente será guardar en MongoDB un campo calculado al momento de insertar:

edad_paciente
grupo_etario

Ejemplo:

niño
joven
adulto
adulto_mayor

Así MongoDB podrá agrupar sin consultar PostgreSQL en cada reporte.

Resultado esperado:

grupo_etario
promedio_presion_sistolica
promedio_presion_diastolica
promedio_frecuencia_cardiaca
promedio_temperatura

Esta será una desnormalización justificada.

Pipeline 4: Tiempo promedio entre consultas por paciente

Responderá:

RC-10. Tiempo promedio entre consultas por paciente.

Usará:

\$setWindowFields
\$shift
\$dateDiff

\$group

Resultado esperado:

id_paciente
promedio_dias_entre_consultas
cantidad_consultas

Este pipeline es más avanzado, pero es defendible y demuestra dominio.

Pipeline con \$facet

El pipeline con \$facet será un reporte clínico consolidado por especialidad.

Se llamará:

pipeline_resumen_clinico_especialidad

Devolverá en una sola ejecución:

1. Top diagnósticos
2. Medicamentos más usados
3. Promedio de signos vitales

Estructura conceptual:

```
db.historiales_clinicos.aggregate([
  {
    $facet: {
      diagnosticos_frecuentes: [
        { $unwind: "$diagnosticos" },
        { $group: { _id: "$diagnosticos.descripcion", total: { $sum: 1 } } },
        { $sort: { total: -1 } },
        { $limit: 5 }
      ],
      medicamentos_frecuentes: [
        { $unwind: "$medicamentos" },
        { $group: { _id: "$medicamentos.nombre", total: { $sum: 1 } } },
        { $sort: { total: -1 } },
      ]
    }
  }
])
```

```

    { $limit: 5 }
  ],
  signos_vitales_promedio: [
    {
      $group: {
        _id: "$especialidad",
        promedio_frecuencia_cardiaca: {
          $avg: "$signos_vitales.frecuencia_cardiaca"
        },
        promedio_temperatura: {
          $avg: "$signos_vitales.temperatura"
        }
      }
    }
  ]
}
}
]);

```

Este será nuestro pipeline más fuerte para defensa.

Nivel MongoDB 8 — Validaciones que haremos

No vamos a confiar solo en que MongoDB inserte documentos. Haremos pruebas.

Validaremos:

1. Insertar historial clínico válido.
2. Rechazar historial sin id_cita.
3. Rechazar historial sin id_paciente.
4. Rechazar historial sin motivo_consulta.
5. Confirmar que diagnósticos y medicamentos sean arrays.
6. Confirmar que el historial se pueda consultar por paciente.
7. Confirmar que los pipelines devuelvan resultados.

La validación de “cita atendida” se hará desde la API consultando PostgreSQL.

Nivel MongoDB 9 — Integración con API Express

Cuando integremos la API, el endpoint será algo así:

POST /api/historiales-clinicos

El flujo correcto será:

1. API recibe datos clínicos.
2. API consulta PostgreSQL para verificar que id_cita existe.
3. API verifica que la cita esté en estado atendida.
4. API obtiene datos básicos: paciente, médico, especialidad.
5. API normaliza los datos con `normalizarHistorialClinico()`.
6. API inserta el documento en MongoDB.
7. API devuelve confirmación.

También tendremos endpoint:

GET /api/historiales-clinicos/paciente/:id_paciente

Este mostrará el historial clínico completo del paciente ordenado por fecha.

Nivel MongoDB 10 — Qué debe quedar al finalizar MongoDB

Cuando terminemos MongoDB, debemos tener:

1. Base clinica_privada_mongo creada.
 2. Colección historiales_clinicos creada.
 3. Documento de estructura de colección.
 4. Justificación de embebido vs referencias.
 5. Función JavaScript `normalizarHistorialClinico()`.
 6. Mínimo 150 historiales clínicos insertados.
 7. Índices creados.
 8. 4 pipelines funcionando.
 9. 1 pipeline con `$facet`.
 10. Pruebas de consultas clínicas.
 11. Explicación lista para defensa.
 12. Integración posterior con API Express.
-

Explicación sencilla para defensa

La defensa de MongoDB será esta:

Usamos PostgreSQL para la parte transaccional del sistema: pacientes, médicos, citas, facturas, pagos y auditoría, porque esos datos necesitan integridad relacional, transacciones y reglas estrictas. Usamos MongoDB para los historiales clínicos porque la información médica cambia según la especialidad. Un cardiólogo, un dermatólogo y un pediatra no registran exactamente los mismos datos. Por eso diseñamos la colección `historiales_clinicos`, donde cada documento representa una consulta atendida y guarda diagnósticos, medicamentos, signos vitales, exámenes y campos específicos de la especialidad. Además, guardamos referencias hacia PostgreSQL mediante `id_cita`, `id_paciente`, `id_medico` e `id_especialidad`.

Orden real cuando iniciemos MongoDB

Cuando retomemos MongoDB, no empezaremos con pipelines. El orden correcto será:

1. Crear base y colección.
2. Definir estructura oficial del documento.
3. Crear schema Mongoose.
4. Crear función normalizadora.
5. Insertar 3 historiales manuales de prueba.
6. Probar consultas básicas.
7. Insertar datos masivos.
8. Crear índices.
9. Crear pipelines.
10. Documentar decisiones.

Ese será el camino más seguro para dejar MongoDB perfecto y defendible.